

One Language.
Infinite Power.
That's
JavaScript!



JAVASCRIPT HANDBOOK



Fundamentals - The Foundation

- ✓ Lightweight
- ✓ Interpreted
- ✓ Dynamic
- ✓ Single Threaded
- ✓ Versatile
- ✓ Cross Platform

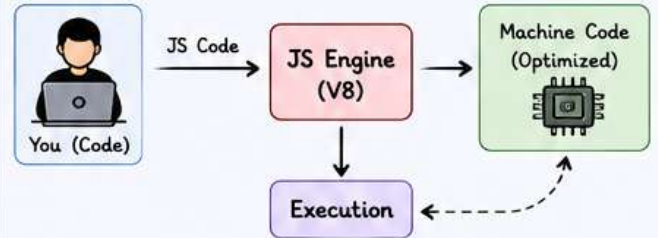
1 WHAT IS JAVASCRIPT?

- ✓ JavaScript is a high-level, interpreted programming language.
- ✓ It makes web pages interactive & dynamic.
- ✓ Runs on the browser (V8 Engine).
- ✓ Used in Frontend, Backend (Node.js), Mobile, Desktop & even AI tools!



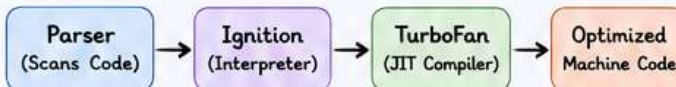
Output → 30

2 HOW JS WORKS?



- ✓ JS Code → Parsed → Compiled → Executed
- ✓ V8 engine (Chrome) is the fastest JS engine

3 JS ENGINE (V8) ARCHITECTURE



- ✓ Parser breaks code into tokens
- ✓ Ignition interprets & runs baseline code
- ✓ TurboFan optimizes hot code (JIT Compilation)
- ✓ Final output is super fast execution



INTERVIEW TIP

Why is JS called **Single Threaded**?

- JS has only one **Call Stack**.
- It uses **Event Loop** to handle async operations.
- This avoids race conditions & keeps things predictable.

4 SINGLE THREADED NATURE

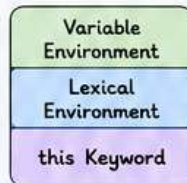
One Call Stack → One Task at a time



Why?

- ✓ Predictable execution
- ✓ Simpler concurrency model
- ✓ Avoid shared memory issues

5 EXECUTION CONTEXT

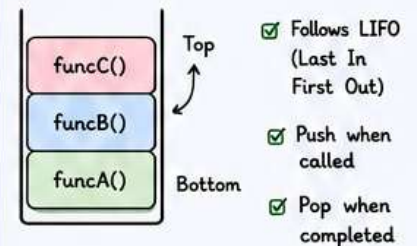


Created every time a function runs.

- Global Context (default)
- Function Context (on invocation)

Example: When a function is called, JS creates a new Execution Context.

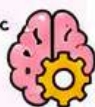
6 CALL STACK



- ✓ Follows LIFO (Last In First Out)
- ✓ Push when called
- ✓ Pop when completed

INTERVIEW INSIGHT

- ✓ JS is synchronous in nature but non-blocking.
- ✓ It uses Event Loop for async tasks.
- ✓ Everything in JS happens inside the Call Stack.



QUICK CODE EXAMPLE

```

console.log('Start');
setTimeout(() => {
  console.log('Async Task');
}, 0);
console.log('End');
    
```

Sync (Immediate)
Async (Event Loop)
Sync (Immediate)

COMMON MYTHS

- JS is multi-threaded ✗
- JS is only for frontend ✗
- setTimeout is inaccurate ✗
- == and === are same ✗
- JS is slow ✗

QUICK RECAP

- ✓ JS is interpreted
- ✓ Runs on V8 Engine
- ✓ Single Threaded
- ✓ Uses Call Stack + Event Loop
- ✓ Execution Context created per run

SAVE THIS POST
for future revision!

COMMENT "JS"
for full PDF in your DM!

SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!



Know your
Data.
Control your
Code.



JAVASCRIPT HANDBOOK

Variables & Data Types



✓ Declarations ✓ Data Types ✓ Type Coercion ✓ Equality ✓ Truthy & Falsy ✓ typeof Operator

1 VAR, LET & CONST

Feature	var	let	const
Scope	Function Scoped	Block Scoped	Block Scoped
Re-declaration	✓ Yes	✗ No	✗ No
Re-assignment	✓ Yes	✓ Yes	✗ No
Hoisting	Hoisted & Init = undefined	Hoisted but in Temporal Dead Zone (TDZ)	Hoisted but in Temporal Dead Zone (TDZ)
Use Case	Old code (avoid)	Values that change	Values that never change

```
var a = 10;
let b = 20;
const c = 30;
// Valid
b = 25;
// Invalid
c = 35; // TypeError: Assignment to constant variable
```

2 DATA TYPES IN JAVASCRIPT

PRIMITIVE (Stored by Value)

123 'abc' true null undefined Symbol()
Number String Boolean Null Undefined Symbol

REFERENCE (Stored by Reference)

{ } [] f(x) Date /abc/
Object Array Function Date RegExp

```
let x = 10; // primitive
let y = x; // copy of value
x = 20;
console.log(y); // 10

let obj1 = { name: 'Abhi' };
let obj2 = obj1; // reference copy
obj2.name = 'TechHub';
console.log(obj1.name); // TechHub
```

MEMORY VISUAL

Primitive (Stack)

x → 10
y → 10

Reference (Heap)

obj1 → { name: 'TechHub' }
obj2 → { name: 'TechHub' }

3 TYPE COERCION (Conversion)

IMPLICIT (Automatic)

- JS automatically converts types when needed.
- Can lead to unexpected results.

```
console.log('5' + 1); // '51'
console.log('5' - 1); // 4
console.log(true + 1); // 2
console.log(null + 1); // 1
console.log(undefined + 1); // NaN
```

EXPLICIT (Manual)

Use built-in functions.

```
String(123) // '123'
Number('123') // 123
Boolean(0) // false
parseInt('123px') // 123
parseFloat('10.5rem') // 10.5
```



PRO TIP

Always be explicit in conversions to avoid bugs.

4 == vs ===

== (Loose Equality)

- Compares after type conversion.
- Only compares value.

```
console.log(5 == '5'); // true
console.log(null == undefined); // true
```

=== (Strict Equality)

- No type conversion.
- Compares value + type.

```
console.log(5 === '5'); // false
console.log(null === undefined); // false
```



BEST PRACTICE

Always use === instead of == to avoid unexpected behavior.

EXAMPLE

```
0 == false // true
0 === false // false
```

5 TYPEOF OPERATOR

Returns the type of the operand.

```
typeof 123; // 'number'
typeof 'hello'; // 'string'
typeof true; // 'boolean'
typeof undefined; // 'undefined'
typeof null; // 'object' (bug in JS)
typeof Symbol(); // 'symbol'
typeof {}; // 'object'
typeof []; // 'object'
typeof function(){}; // 'function'
```

NOTE

typeof null returns 'object' which is a legacy bug in JS.



6 TRUTHY vs FALSY

FALSY VALUES (Only 7)

- false
- 0, -0, 0n
- '' (empty string)
- null
- undefined
- NaN
- document.all (legacy)

Everything else is TRUTHY

- '0', 'false', ''
- [] (empty array)
- {} (empty object)
- function(){}
- any non-zero number
- any non-empty string

```
if ('hello') console.log('Truthy'); // Executes
if ('') console.log('Falsy'); // Not Executed
if ([]) console.log('Truthy'); // Executes
if (0) console.log('Falsy'); // Not Executed
```



INTERVIEW INSIGHT

Understanding coercion, equality and falsy values is key to avoid subtle bugs in real world applications.

COMMON MISTAKES

- ✗ Using var in modern JS
- ✗ Comparing with ==
- ✗ Ignoring Type Coercion
- ✗ Assuming [] or {} is falsy



REMEMBER

- Use const by default
- Use let when values change
- Use === always
- Know your data!



INTERVIEW TIP

Practice writing output of these snippets on paper. Interviewers love these questions!

QUICK RECAP

- ✓ Variables → var, let, const
- ✓ Data Types → Primitive & Reference
- ✓ Conversions → Implicit & Explicit
- ✓ Equality → == vs ===
- ✓ Truthy / Falsy → 7 Falsy values



SAVE THIS POST
for future revision!



COMMENT "JS"
for full PDF in your DM!



SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!



Functions
Make Code
Reusable.
Scope Keeps it
Organized! 😊



JAVASCRIPT HANDBOOK

Functions & Scope

Understand Scope, Master Closures



✓ Reusable Code

✓ Better Structure

✓ Scope Control

✓ Closures

✓ Hoisting

✓ Maintainable

1 TYPES OF FUNCTIONS

Function Declaration

```
function add(a, b) {
  return a + b;
}

add(2, 3); // 5
```

- ✓ Hoisted
- ✓ Can call before declaration

Function Expression

```
const add = function(a, b) {
  return a + b;
};

add(2, 3); // 5
```

- ✓ Not hoisted
- ✓ Must call after declaration

Arrow Function (ES6+)

```
const add = (a, b) => a + b;

add(2, 3); // 5
```

- ✓ Short & clean syntax
- ✓ Lexical this (no own this)

2 SCOPE & SCOPE CHAIN

```
let a = 10; // Global

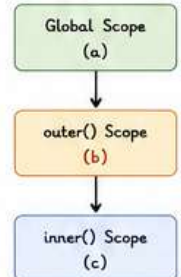
function outer() {
  let b = 20; // Outer

  function inner() {
    let c = 30; // Inner
    console.log(a + b + c); // 60
  }
  inner();
}

outer();
```

💡 Inner function can access its own scope, outer scope & global scope.

SCOPE CHAIN DIAGRAM



3 HOISTING IN JAVASCRIPT

VARIABLE HOISTING

```
console.log(x); // undefined
var x = 5;
```

Only declaration is hoisted, initialization stays in place.

FUNCTION HOISTING

```
sayHello(); // Works
function sayHello() {
  console.log('Hello!');
}
```

Function declarations are fully hoisted.

LET / CONST HOISTING (TDZ)

```
console.log(y); // ReferenceError
let y = 10;
const z = 20;
```

Not accessible before initialization (Temporal Dead Zone).

⚠ Remember: var is hoisted & initialized as undefined. let & const are hoisted but not initialized (TDZ).

4 WHAT IS CLOSURE?

A closure is a function that remembers the variables from its outer (enclosing) scope even after the outer function has finished executing.

```
function outerFn() {
  let count = 0; // private variable

  return function innerFn() {
    count++;
    console.log(count);
  };
}

const counter = outerFn();
counter(); // 1
counter(); // 2
counter(); // 3
```

KEY POINTS

- ✓ Inner function has access to outer function's variables.
- ✓ Variables don't get destroyed.
- ✓ Useful for data privacy & function factories.

5 CLOSURES IN DETAIL

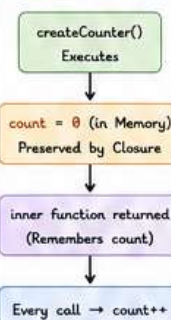
EXAMPLE: COUNTER FUNCTION

```
function createCounter() {
  let count = 0;

  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

HOW IT WORKS?



ANOTHER EXAMPLE: PRIVATE DATA

```
function bankAccount(balance) {
  return {
    getBalance: function() {
      return balance;
    },
    deposit: function(amount) {
      balance += amount;
    }
  };
}

const account = bankAccount(1000);
account.deposit(500);
console.log(account.getBalance());
// 1500
```

CLOSURE USE CASES

- 🔒 Data Privacy
- 🏭 Function Factories
- 🕒 Memoization
- ⚙ Event Handlers
- 🧩 Module Patterns

INTERVIEW TIP 💡

A closure gives you access to an outer function's scope from an inner function, even after the outer function has returned.

COMMON MISTAKE ❌

Forgetting var in loops with closures.

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
} // Output: 3 3 3

Fix with let:
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
} // Output: 0 1 2
```

6 ARROW FUNCTIONS & THIS

REGULAR FUNCTION

```
const obj = {
  value: 100,
  getValue: function() {
    return this.value;
  }
};

console.log(obj.getValue());
// 100
```

ARROW FUNCTION (LEXICAL THIS)

```
const obj = {
  value: 100,
  getValue: () => {
    return this.value;
  }
};

console.log(obj.getValue());
// undefined (this is not obj)
```

KEY DIFFERENCE

Feature	Regular Function	Arrow Function
this binding	Dynamic	Lexical (inherited)
arguments	Available	Not available
Constructor	Can be used	Cannot be used
Best for	Methods, objects	Callbacks, short fn

PRO TIP 🚀

Use Arrow functions for callbacks (map, filter, reduce, setTimeout, etc.) to avoid this confusion.

REMEMBER ⭐

- ✓ Functions = Reusable blocks of code
- ✓ Scope = Visibility of variables
- ✓ Hoisting = JS moves declarations up
- ✓ Closure = Remember outer scope

QUICK RECAP

fx 3 Types of Functions

🔗 Scope Chain (inner → outer → global)

↑ Hoisting (Know what is lifted)

🔒 Closures (Remember & preserve)

➡ Arrow Functions (Lexical this)

📖 SAVE THIS POST for future revision!

💬 COMMENT "JS" for full PDF in your DM!

🚀 SHARE IT with your friends!

Follow @abhi_techhub for more such content!



Data in
Structure,
Power in
Your Code!



JAVASCRIPT HANDBOOK

Objects & Arrays



Organized Data Powerful Methods Clean Code Less Code More Readable Developer Superpower

1 OBJECTS BASICS

Objects store key-value pairs.

```
const user = {
  id: 101,
  name: 'Abhi',
  role: 'Developer',
  isActive: true,

  greet() {
    return `Hello ${this.name}`;
  }
};

console.log(user.name); // Abhi
console.log(user.greet()); // Hello Abhi
```

ACCESSING

- Dot Notation
user.name
- Bracket Notation
user['role']
- Add / Update
user.city = 'Pune';
- Delete
delete user.role;

TIP

Objects are mutable (changeable). Values are stored by reference.

2 DESTRUCTURING

Extract values from Objects or Arrays easily.

Object Destructuring

```
const user = {
  id: 101,
  name: 'Abhi',
  city: 'Pune'
};

const { id, name, city } = user;
console.log(name); // Abhi
```

Rename + Default

```
const { name: username, city,
  country = 'India' } = user;
console.log(username); // Abhi
console.log(country); // India
```

Array Destructuring

```
const arr = [10, 20, 30, 40];
const [a, b, c, d] = arr;
console.log(b, d); // 20 40
```

Skip Values

```
const [, , d] = arr;
console.log(a, d); // 10 40
```

3 SPREAD OPERATOR (...)

Expands elements of an iterable (Array / Object).

Arrays

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2);
// [1, 2, 3, 4, 5]
```

Objects

```
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 };
console.log(obj2);
// { a: 1, b: 2, c: 3 }
```

Use Case

- ✓ Copy arrays / objects
- ✓ Merge arrays / objects
- ✓ Pass multiple arguments to functions

```
function sum(a, b, c) {
  return a + b + c;
}

const nums = [1, 2, 3];
console.log(sum(...nums)); // 6
```

4 REST OPERATOR (...)

Collects elements into a single array / object.

Functions

```
function sumAll(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}

console.log(sumAll(1, 2, 3, 4, 5));
// 15
```

Destructuring

```
const [first, ...rest] = [10, 20, 30, 40];
console.log(first); // 10
console.log(rest); // [20, 30, 40]
```

Use Cases

- ✓ Handle unlimited arguments in functions
- ✓ Collect remaining items in arrays
- ✓ Ignore unneeded values

NOTE

Spread expands, Rest collects.

5 POPULAR ARRAY METHODS

map()

Transforms each element.

```
const nums = [1, 2, 3];
const sq = nums.map(n => n * n);
// [1, 4, 9]
```

[1, 2, 3]
↓ map(n*n)
[1, 4, 9]

filter()

Filters elements based on condition.

```
const nums = [1, 2, 3, 4];
const even = nums.filter(n => n % 2 === 0);
// [2, 4]
```

[1, 2, 3, 4]
↓ n % 2 === 0
[2, 4]

reduce()

Reduces to a single value.

```
const nums = [1, 2, 3];
const sum = nums.reduce((acc, n) => acc + n, 0);
// 6
```

[1, 2, 3]
↓ reduce(+)
6

find()

Returns first element that matches.

```
const users = [{id:1}, {id:2}];
const u = users.find(u => u.id === 2);
// {id: 2}
```

[[{1}, {2}, {3}]]
↓ id === 2
{ id: 2 }

some()

Checks if any element matches.

```
const nums = [1, 2, 3, 4];
const hasEven = nums.some(n => n % 2 === 0);
// true
```

[1, 2, 3, 4]
↓ n % 2 === 0 (any)
true

every()

Checks if all elements match.

```
const nums = [2, 4, 6];
const allEven = nums.every(n => n % 2 === 0);
// true
```

[2, 4, 6]
↓ n % 2 === 0 (all)
true

INTERVIEW INSIGHT

- Arrays are objects in JS.
- Objects are reference types.
- Spread / Rest make code shorter & cleaner.
- Master these → Write modern, clean JS!

COMMON MISTAKES

- ✗ Forgetting that objects are by reference.
- ✗ Mutating original arrays while using methods.
- ✗ Confusing map() with forEach().
- ✗ Overusing loops instead of built-in methods.

PRO TIP

- ✓ Use map/filter/reduce instead of loops.
- ✓ Use destructuring to write clean code.
- ✓ Spread for copying, Rest for grouping.

QUICK RECAP

- ✓ Objects → Key-Value pairs
- ✓ Destructuring → Extract values
- ✓ Spread → Expand
- ✓ Rest → Collect
- ✓ Array Methods → Write less, do more!

SAVE THIS POST
for future revision!

COMMENT "JS"
for full PDF in your DM!

SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!

Async is
where JS
becomes
Powerful! ⚡
😊

JS

JAVASCRIPT HANDBOOK

Async JavaScript

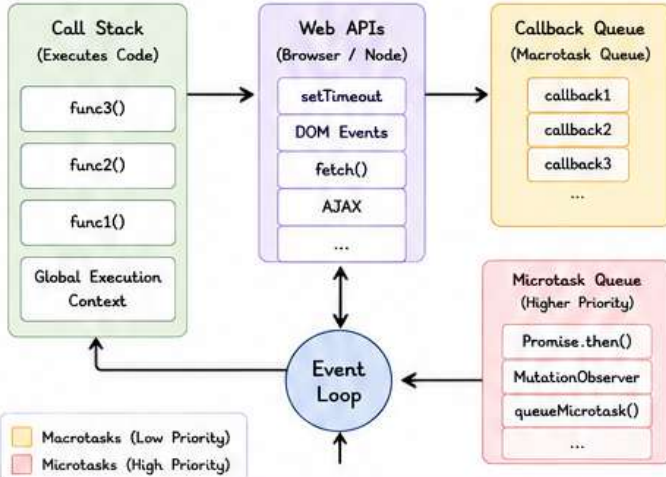
Event Loop, Promises, Async/Await & More



✔ Event Loop ✔ Callbacks ✔ Promises ✔ Async/Await ✔ Microtasks ✔ Macrotasks ✔ Concurrency

1 THE EVENT LOOP

The Event Loop is the heart of Async JavaScript.



★ Flow: 1) Call Stack → 2) Microtasks → 3) Macrotasks → Repeat

2 MACROTASKS vs MICROTASKS

MACROTASKS (Low Priority Queue)

- setTimeout
- setInterval
- setImmediate (Node)
- I/O Operations
- UI Rendering
- Message Channel

MICROTASKS (High Priority Queue)

- Promise.then()
- .catch(), .finally()
- async/await
- MutationObserver
- queueMicrotask()

KEY POINTS

- ✔ Microtasks are executed first, before the next macrotask.
- ✔ Microtasks run to completion.
- ✔ If microtasks are continuously added, macrotasks may be delayed.

```
console.log('Start'); // 1
setTimeout(() => console.log('setTimeout'), 0); // 4
Promise.resolve().then(() => console.log('Promise')); // 2
console.log('End'); // 3
```

// Output: Start → End → Promise → setTimeout

3 CALLBACKS

Old school way to handle async operations.

```
function getData(callback) {
  setTimeout(() => {
    callback('Data Received');
  }, 1000);
}

getData((data) => {
  console.log(data);
  getData((data2) => {
    console.log(data2);
    getData((data3) => {
      console.log(data3); // Callback Hell 😱
    });
  });
});
```

⚠ Callback Hell leads to unreadable & hard to maintain code.

4 PROMISES

Promises represent the eventual completion (or failure) of an async operation.

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    let success = true;
    if (success) resolve('Success!');
    else reject('Something went wrong!');
  }, 1000);
});

promise
  .then((result) => console.log(result))
  .catch((error) => console.error(error))
  .finally(() => console.log('Done!'));
```

STATES OF A PROMISE

Pending → Fulfilled (Resolved) → Rejected

5 ASYNC / AWAIT

Modern & clean way to write async code.

```
async function getData() {
  try {
    const res = await fetch('https://api.example.com/data');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error('Error:', err);
  } finally {
    console.log('Request Completed');
  }
}

getData();
```

PRO TIP

Always use try...catch with async/await to handle errors gracefully.

6 SETTIMEOUT & SETINTERVAL

setTimeout

```
setTimeout(() => {
  console.log('Runs once after 2s');
}, 2000);
```

setInterval

```
const id = setInterval(() => {
  console.log('Runs every 1s');
}, 1000);

clearInterval(id); // stop
```



NOTE

Timers are Macrotasks. They wait until the Call Stack is empty.

7 PROMISE UTILITY METHODS

Method	Description	Example
Promise.all()	Resolves when all promises resolve	Wait for all
Promise.allSettled()	Resolves when all promises settle	Success or fail
Promise.race()	Resolves/rejects as soon as one promise settles	First one wins
Promise.any()	Resolves when any one promise resolves	First success
Promise.resolve()	Returns a resolved promise	Promise.resolve(5)
Promise.reject()	Returns a rejected promise	Promise.reject(err)

8 EXECUTION ORDER EXAMPLE

```
console.log('1 Start');
setTimeout(() => console.log('2 setTimeout'), 0);
Promise.resolve().then(() => console.log('3 Promise'));
queueMicrotask(() => console.log('4 queueMicrotask'));
console.log('5 End');
```

OUTPUT ORDER

- 1 Start → 5 End → 4 queueMicrotask → 3 Promise → 2 setTimeout

Why?

- Synchronous code runs first.
- Then Microtasks (queueMicrotask, Promise).
- Then Macrotasks (setTimeout).

INTERVIEW INSIGHT

- Event Loop is crucial for understanding async behavior.
- Microtasks have higher priority.
- Promises & async/await help avoid callback hell.



COMMON MISTAKES

- Forgetting to return in .then() chains
- Not handling promise rejections
- Blocking the event loop with heavy synchronous code
- Creating infinite microtask loop



BEST PRACTICE

- Prefer async/await over .then()
- Handle errors using try/catch
- Keep tasks small & non-blocking
- Use Promise.all for parallel calls

QUICK RECAP

- ✔ Event Loop = Call Stack + Queues + Loop
- ✔ Microtasks > Macrotasks
- ✔ Promise = Pending → Fulfilled / Rejected
- ✔ Async/Await = Syntactic Sugar over Promises



SAVE THIS POST
for future revision!

COMMENT "JS"
for full PDF in your DM!

SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!

DOM is the Bridge between JavaScript & the Web Page!

JS

JAVASCRIPT HANDBOOK

DOM & EVENTS

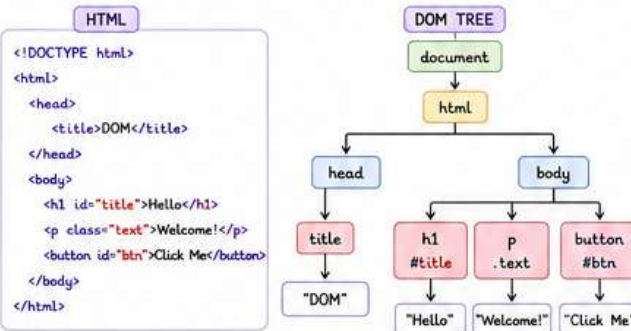
Interact with the Web. Make it Dynamic.

PAGE 6

DOM Tree Select Elements Events Event Propagation Delegation Debounce Throttle

1 THE DOM TREE

The DOM represents the HTML document as a tree of objects.



Everything in the page is a node (element, text, attribute, comment, etc.)

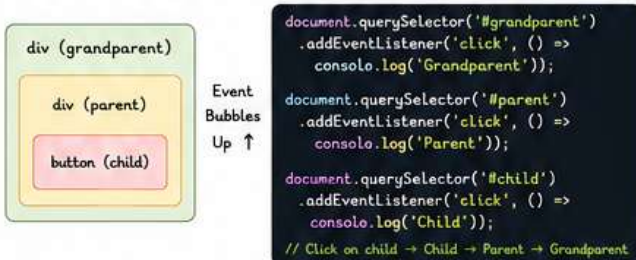
2 SELECTING ELEMENTS

getElementById()	Select by id (returns single element) <code>document.getElementById('btn');</code>
getElementsByClassName()	Select by class (returns HTMLCollection) <code>document.getElementsByClassName('text');</code>
getElementsByTagName()	Select by tag (returns HTMLCollection) <code>document.getElementsByTagName('p');</code>
querySelector()	Select first match (CSS selector) <code>document.querySelector('.text');</code>
querySelectorAll()	Select all matches (NodeList) <code>document.querySelectorAll('p.text');</code>

PRO TIP Use querySelector / querySelectorAll (modern & powerful).

3 EVENT BUBBLING

Event starts from the target and bubbles up to the parent elements.



Bubbling is the default behavior in DOM.

4 EVENT CAPTURING

Event starts from the root and goes down to the target.



Add true as the third parameter to enable capturing.

5 EVENT DELEGATION

Attach a single event listener to a parent to handle events on its children.



- Less code
- Better performance
- Works for dynamic elements

6 DEBOUNCE vs THROTTLE

DEBOUNCE	THROTTLE
Executes only after the event stops firing.	Executes at regular intervals no matter how many events fire.
Events: (typing)	Events: (scrolling)
Execute: ★	Execute: ★ ★ ★
<pre>function debounce(fn, delay) { let timer; return function(...args) { clearTimeout(timer); timer = setTimeout(() => fn(...args), delay); }; }</pre>	<pre>function throttle(fn, delay) { let last = 0; return function(...args) { const now = Date.now(); if (now - last > delay) { last = now; fn(...args); } }; }</pre>

Debounce → Search box, resize, input
Throttle → Scroll, mousemove, infinite scroll

7 DEBOUNCE & THROTTLE EXAMPLES

Debounce Example (Search Box)

```
const search = debounce(() => {
  console.log('Searching...');
}, 500);

input.addEventListener('input', search);
```

Throttle Example (Scroll Event)

```
const handleScroll = throttle(() => {
  console.log('Scroll Event');
}, 1000);

window.addEventListener('scroll', handleScroll);
```

NOTE

Use debounce when you want to wait.

Use throttle when you want to control rate.



INTERVIEW INSIGHT

- Know the Event Loop
- Understand Bubbling vs Capturing
- Use Delegation for performance
- Debounce vs Throttle - big favorite!

COMMON MISTAKES

- Attaching events to many elements
- Not using cleanup on listeners
- Using debounce for continuous tasks
- Forgetting to use true for capturing

BEST PRACTICE

- Event Delegation over direct binding
- Use passive: true for scroll/touch
- Remove listeners to avoid memory leaks
- Write clean, reusable functions

QUICK RECAP

- Select → querySelector
- Events → addEventListener
- Bubbling ↑ Capturing ↓
- Delegate → Parent listener
- Debounce → Wait Throttle → Control

SAVE THIS POST for future revision!

COMMENT "JS" for full PDF in your DM!

SHARE IT with your friends!

Follow @abhi_techhub for more such content!

Practice DOM. Master Events. Build Amazing Experiences!

Modern JS
makes code
Shorter, Cleaner
& Powerful!

JS

JAVASCRIPT HANDBOOK

ES6+ Features

Modern Syntax. Cleaner Code. Better Developer.



Modern Syntax Cleaner Code Less Boilerplate Better Readability Powerful Features Future Ready

1 TEMPLATE LITERALS

Multi-line strings, interpolation & expressions.

```
const name = 'Abhi';
const age = 23;

const msg = `Hello ${name} 🙋
You are ${age} years old.
Today is ${new Date().toLocaleDateString()}`;

console.log(msg);
```

Output

Hello Abhi 🙋
You are 23 years old.
Today is Fri May 17 2025.



TIP

Use backticks ` instead of quotes.
Supports multi-line & expressions.

2 MODULES (IMPORT / EXPORT)

Split code into reusable modules.

math.js

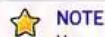
```
export const add = (a, b) => a + b;
export const PI = 3.14159;

export default function mul(a, b) {
  return a * b;
}
```

app.js

```
import mul, { add, PI } from './math.js';

console.log(add(2, 3)); // 5
console.log(mul(2, 3)); // 6
console.log(PI); // 3.14159
```



NOTE

Use named exports for multiple values.
Use default export for one main thing.

3 OPTIONAL CHAINING (?.)

Safely access nested properties.

```
const user = {
  name: 'Abhi',
  address: {
    city: 'Pune'
  }
};

console.log(user.address?.city); // Pune
console.log(user.profile?.bio); // undefined
```

WHY USE IT?

Avoids "Cannot read properties of undefined" errors in deep structures.

4 NULLISH COALESCING (??)

Returns right value only when left is null or undefined.

```
const name = null ?? 'Guest'; // Guest
const age = 0 ?? 18; // 0 (0 is valid)
const city = undefined ?? 'Pune'; // Pune
const value = '' ?? 'Default'; // '' (empty is valid)
```

?? vs ||

|| (OR)

Returns right if left is
Falsy (false, 0, '', null,
undefined, NaN)

?? (Nullish Coalescing)

Returns right only if left is
null or undefined



TIP: Prefer ?? over || to handle valid falsy values.

5 DEFAULT PARAMETERS

Set default values for function parameters.

```
function greet(name = 'Guest', msg = 'Hello!') {
  return `${msg} ${name}`;
}

console.log(greet('Abhi')); // Hello! Abhi
console.log(greet(undefined, 'Hi')); // Hi Guest
console.log(greet()); // Hello! Guest
```



PRO TIP

Defaults make functions flexible and
reduce undefined checks.

6 DESTRUCTURING

Extract values from arrays or objects.

Array Destructuring

```
const nums = [10, 20, 30];
const [a, b, c] = nums;
// a = 10, b = 20, c = 30
```

Object Destructuring

```
const user = { id: 1, name: 'Abhi', role: 'Dev' };
const { id, name, role } = user;
// id = 1, name = 'Abhi', role = 'Dev'
```



Makes code shorter & more readable.

7 SPREAD OPERATOR (...)

Expand iterable elements.

For Arrays

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]
```

For Objects

```
const user = { name: 'Abhi', age: 23 };
const copy = { ...user, city: 'Pune' };
// { name: 'Abhi', age: 23, city: 'Pune' }
```



Great for copying, merging & passing arguments.

8 REST OPERATOR (...)

Collect remaining elements into an array or object.

In Functions

```
function sum(...nums) {
  return nums.reduce((acc, n) => acc + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

In Objects

```
const { id, name, ...rest } = user;
// rest = { age: 23, city: 'Pune' }
```



Use Rest to handle unknown number of
arguments or remaining properties.

9 OTHER COOL FEATURES

- Short Circuit (&&, ||)
console.log(true && 'Yes'); // 'Yes'
console.log(false || 'No'); // 'No'
- Exponentiation Operator (**)
console.log(2 ** 3); // 8
- Array.find()
const arr = [1, 2, 3, 4];
console.log(arr.find(n => n > 2)); // 3
- Array.includes()
console.log([1, 2, 3].includes(2)); // true
- String.includes()
console.log('hello'.includes('ell')); // true
- Object.entries()
console.log(Object.entries({a:1,b:2}));
// [['a',1], ['b',2]]

INTERVIEW INSIGHT

- ES6+ is widely used in modern codebase.
- Helps write cleaner, shorter & more maintainable code.
- Most features are asked in frontend interviews.



COMMON MISTAKES

- ✗ Overusing destructuring (hurts readability sometimes).
- ✗ Confusing || with ??.
- ✗ Forgetting that spread creates shallow copies.



BEST PRACTICE

- ✓ Use const by default.
- ✓ Prefer modern syntax for better readability.
- ✓ Keep code simple, clean & consistent.



QUICK RECAP

- ✓ Template Literals
- ✓ Modules (Import/Export)
- ✓ Optional Chaining (?.)
- ✓ Nullish Coalescing (??)
- ✓ Default Parameters
- ✓ Destructuring
- ✓ Spread & Rest Operators

REMEMBER

"Modern JavaScript is not about writing more code, it's about writing better code." 😊



SAVE THIS POST
for future revision!



COMMENT "JS"
for full PDF in your DM!



SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!



Understand
HOW JS works
under the hood.
This is where
Senior devs shine! ✨
😊



JAVASCRIPT HANDBOOK

Advanced JavaScript

Pro Level Concepts. Write Better Code.



Deep Concepts

Better Thinking

Write Efficient Code

Crack Interviews

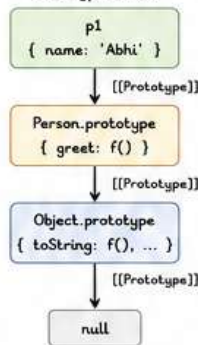
Build like a Pro

1 PROTOTYPE & PROTOTYPE CHAIN

Every object in JS has a hidden property `[[Prototype]]`.

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function () {
  return 'Hello ' + this.name;
};
const p1 = new Person('Abhi');
console.log(p1.greet()); // Hello Abhi
```

Prototype Chain



TIP

When a property/method is not found in an object, JS looks into its prototype chain.

2 THE 'this' KEYWORD

Value of this depends on how a function is called.

Context	Example	this refers to
Global	<code>console.log(this)</code>	window (browser) { } (strict mode in Node)
Object Method	<code>obj.fn()</code>	obj
Constructor	<code>new Fn()</code>	new object
Event Handler	<code>btn.onclick = fn</code>	DOM element
Explicit Binding	<code>call / apply / bind</code>	manually set



Remember:

this is not fixed. It depends on the call site, not where the function is written.

3 call(), apply() & bind()

Methods to explicitly set the value of this.

call()

Pass arguments individually.

```
function greet(city){
  return 'Hi from ' + city;
}
greet.call({name: 'Abhi'}, 'Pune');
// Hi from Pune
```

apply()

Pass arguments as array.

```
greet.apply({name: 'Abhi'}, ['Pune']);
// Hi from Pune
```

bind()

Returns a new bound function.

```
const g = greet.bind({name: 'Abhi'}, 'Pune');
console.log(g());
// Hi from Pune
```



Use `call()` when args are separate, `apply()` for array, `bind()` when you want a new function.

4 CURRYING

Transform a function with multiple args into a sequence of functions.

```
function add(a){
  return function(b){
    return function(c){
      return a + b + c;
    };
  };
}
console.log(add(2)(3)(4)); // 9
```

add(2)

```
function (b){
  return function (c){
    return 2 + b + c;
  }
}
```

(3)

```
function (c){
  return 2 + 3 + c;
}
```

(4)

9



PRO TIP

Useful for function composition, clean code & partial application.

5 MEMOIZATION

Cache the results of expensive function calls.

```
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key]) return cache[key];
    const result = fn(...args);
    cache[key] = result;
    return result;
  };
}
const square = memoize(n => n * n);
console.log(square(5)); // 25 (expensive call)
console.log(square(5)); // 25 (from cache)
```



NOTE

Great for performance optimization in recursion, calculations, API responses etc.

6 DEEP COPY vs SHALLOW COPY

Shallow Copy (copies reference)

```
const obj1 = { a: 1, b: { c: 2 } };
const obj2 = { ...obj1 };
obj2.b.c = 99;
console.log(obj1.b.c); // 99 (changed!)
```

Deep Copy (copies actual value)

```
const obj1 = { a: 1, b: { c: 2 } };
const obj2 = JSON.parse(JSON.stringify(obj1));
obj2.b.c = 99;
console.log(obj1.b.c); // 2 (not changed)
```

When to use?

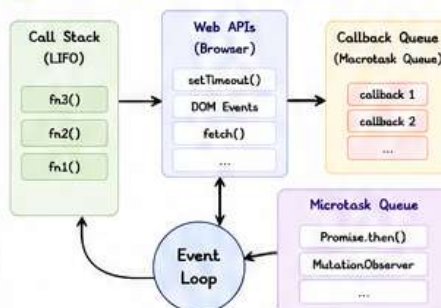
- Shallow: simple objects, performance matters
- Deep: nested objects/arrays, data isolation



PRO TIP

Use `structuredClone()` for deep clone (where supported).

7 EVENT LOOP - THE ENGINE ROOM



Key Order

- Execute all code in Call Stack
- When stack is empty → pick first task from Microtask Queue
- Execute all microtasks
- Then pick first task from Macrotask Queue
- Repeat...

★ Microtasks always have higher priority than Macrotasks.

8 OTHER ADVANCED TOPICS (QUICK LOOK)



Closures

Function remembers its outer scope even after outer function has finished.



Garbage Collection

Memory is freed automatically (unreachable objects).



Hoisting

Variable & function declarations are moved to the top.



IIFE

`(function(){ ... })();`
Creates private scope.



WeakMap / WeakSet

Hold weak references to objects (avoids memory leaks).



Symbol

Unique & immutable value, used as object keys.



Generator Functions

Pause & resume execution (yield keyword).



BigInt

Handle very large integers (> Number limit).

INTERVIEW INSIGHT

- Prototype & this are interview favorites
- Understand Event Loop deeply
- Know call, apply, bind by heart
- Practice Currying & Memoization

COMMON MISTAKES

- Using `==` instead of `===`
- Forgetting context of this
- Shallow copy for nested objects
- Ignoring async behavior

BEST PRACTICE

- Write pure & small functions
- Avoid global scope pollution
- Use `const` by default
- Optimize only where needed

QUICK RECAP

- Prototype & this
- call / apply / bind
- Curry & Memoize
- Deep Copy
- Event Loop
- Advanced Topics

REMEMBER

"JavaScript is not about writing more code. It's about writing better code."



SAVE THIS POST
for future revision!



COMMENT "JS"
for full PDF in your DM!



SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!



Fast apps
create happy
users. Optimize
like a Pro! 🚀
😊



JAVASCRIPT HANDBOOK

Browser, Storage & Performance

Make Your App Fast, Secure & Efficient.



- Web Storage
- Cookies
- Memory & GC
- Performance Tips
- Lazy Loading
- Code Splitting
- Best Practices

1 WEB STORAGE

Store key-value pairs in the browser.

localStorage

- No expiration
- Data persists even after browser is closed
- Size ~ 5-10MB

sessionStorage

- Data lasts for the page session
- Cleared when tab is closed
- Size ~ 5-10MB

```
// localStorage
localStorage.setItem('user', JSON.stringify({name: 'Abhi'}));
const user = JSON.parse(localStorage.getItem('user'));
localStorage.removeItem('user');
localStorage.clear();

// sessionStorage
sessionStorage.setItem('token', 'abc123');
const token = sessionStorage.getItem('token');
sessionStorage.removeItem('token');
```



TIP:

Always store strings. Use JSON for objects.

2 COOKIES

Small pieces of data sent with every request.

- Stored by browser
- Sent to server automatically
- Size ~ 4KB per cookie

```
// Set Cookie (expires in 7 days)
document.cookie = "user=Abhi; max-age=" +
60*60*24*7 + "; path=/";

// Read Cookies
console.log(document.cookie);

// Delete Cookie
document.cookie = "user=; max-age=0; path=/";
```



NOTE

- Use cookies for auth tokens (With HttpOnly)
- Never store sensitive data in cookies
- Use Secure flag on HTTPS

3 MEMORY LEAKS

Memory that is not released when no longer needed.

Common Causes

- Global variables
- Unremoved event listeners
- Timers (setInterval / setTimeout)
- Closures holding large data
- Detached DOM elements

Example (Leaky Code)

```
function init(){
  const btn = document.getElementById('btn');
  btn.addEventListener('click', () => {
    console.log('Clicked');
  });
} // if init() called again & again -> leak
```



TIP

- Remove listeners when not needed
- Clear timers
- Avoid unnecessary global variables

4 GARBAGE COLLECTION (GC)

JS automatically frees memory that is no longer reachable.

How it works?

- JS engine tracks objects in memory
- If no reference points to an object -> it is removed (garbage collected)



Example

```
let user = { name: 'Abhi' };
user = null; // no reference
// GC will free memory later
```

- Good Practice: Avoid circular references and unnecessary closures.

5 LAZY LOADING

Load resources only when needed.

HTML (Image)

```

```

JavaScript (Intersection Observer)

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if(entry.isIntersecting){
      entry.target.src = entry.target.dataset.src;
      observer.unobserve(entry.target);
    }
  });
});
document.querySelectorAll('img[data-src]').forEach(img => {
  observer.observe(img);
});
```

- Improves performance by loading only visible content.

6 CODE SPLITTING

Break your code into smaller chunks.

Example with Dynamic Import

```
// main.js
console.log('App Started');

// Load module only when needed
button.addEventListener('click', async () => {
  const module = await import('./module.js');
  module.sayHello();
});
```

Benefits

- Smaller initial bundle size
- Faster initial page load
- Better user experience

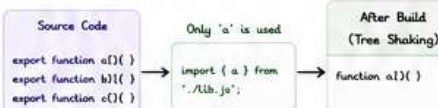


7 PERFORMANCE OPTIMIZATION TIPS

Minimize Reflows & Repaints	Batch DOM reads and writes.
Debounce / Throttle	Limit frequency of expensive operations.
Avoid Long Tasks	Keep tasks < 50ms for smooth UI.
Use Passive Listeners	Improves scrolling performance.
Optimize Images	Use correct size & modern formats (webp).
Use CSS for Animations	Leverage GPU for smooth performance.
Remove Unused Code	Keep your bundle lean.

8 TREE SHAKING

Eliminate unused code during build time.



How it works?

- ES6 modules have static structure.
- Bundlers detect unused exports.
- Unused code is removed from the final bundle.



9 BEST PRACTICES

- Write clean, readable & modular code.
- Use strict mode: 'use strict';
- Prefer const over let, avoid var.
- Avoid blocking the main thread.
- Handle errors gracefully (try...catch).
- Use meaningful variable & function names.
- Keep functions small & focused.
- Test your code & handle edge cases.
- Profile & measure before optimizing.



INTERVIEW INSIGHT

- Know Web Storage vs Cookies.
- Understand Memory Leaks & GC.
- Lazy Loading, Code Splitting = Performance Boost.
- Always think about User Experience!



COMMON MISTAKES

- Storing sensitive data in localStorage
- Forgetting to remove event listeners
- Creating too many global variables
- Not clearing timers
- Ignoring performance profiling



PRO TIP

- Measure -> Don't Guess
- Use DevTools Performance Tab
- Optimize Critical Path First
- Speed = Happy Users



QUICK RECAP

- Storage -> localStorage / cookies
- Memory -> Leaks, GC
- Performance -> Lazy Load, Split
- Best Practices -> Clean, Secure

REMEMBER

"Performance is not just about speed. It's about creating a smooth experience for your users." 😊



SAVE THIS POST
for future revision!



COMMENT "JS"
for full PDF in your DM!



SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!



Great code handles errors like a PRO & keeps users HAPPY! 😊



JAVASCRIPT HANDBOOK

Asynchronous JS, Error Handling & Modern Best Practices

Write Resilient Code. Handle Like a Pro.



✓ Async Code ✓ Promises ✓ Async/Await ✓ Error Handling ✓ Debugging ✓ Best Practices ✓ Keep Improving

1 CALLBACK HELL

Too many nested callbacks = hard to read & maintain.

```
doTask1(() => {
  doTask2(() => {
    doTask3(() => {
      doTask4(() => {
        console.log('Done!');
      });
    });
  });
});
```



Hard to read, handle errors & maintain.
Use Promises or Async/Await instead.

2 PROMISES - THE SOLUTION

Represent the eventual completion (or failure) of a task.

```
doTask1()
  .then(res1 => doTask2(res1))
  .then(res2 => doTask3(res2))
  .then(res3 => doTask4(res3))
  .then(() => console.log('Done!'))
  .catch(err => console.error(err));
```

- ✓ Cleaner than callbacks
- ✓ Easy error handling
- ✓ Chainable

3 ASYNC / AWAIT - THE BEST WAY

Syntactic sugar over Promises.

```
async function doAll() {
  try {
    const res1 = await doTask1();
    const res2 = await doTask2(res1);
    const res3 = await doTask3(res2);
    await doTask4(res3);
    console.log('Done!');
  } catch (err) {
    console.error('Something went wrong:', err);
  }
}
doAll();
```



Looks like sync code, works async.
Use try...catch for error handling.

4 ERROR HANDLING

Always handle errors gracefully.

With Promises

```
fetch('/api/data')
  .then(res => {
    if(!res.ok) throw new Error('API Error');
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error(err.message));
```

With Async/Await

```
try {
  const res = await fetch('/api/data');
  if(!res.ok) throw new Error('API Error');
  const data = await res.json();
  console.log(data);
} catch (err) {
  console.error(err.message);
}
```



TIP:

- Always handle network errors
- Validate response before using
- Show meaningful error messages to users

5 PROMISE METHODS

Useful built-in methods.

Method	Description
Promise.all()	Waits for all promises, rejects if any one fails.
Promise.allSettled()	Waits for all, regardless of result (fulfilled/rejected).
Promise.race()	Resolves/rejects as soon as one promise settles.
Promise.any()	Resolves as soon as any one promise fulfills.

Example: Promise.all()

```
Promise.all([fetch(u1), fetch(u2)])
  .then(res => Promise.all(res.map(r => r.json())))
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

6 FETCH API - BEST PRACTICES

Modern way to make HTTP requests.

```
async function getUsers() {
  try {
    const res = await fetch('https://api.example.com/users');
    if (!res.ok) throw new Error('Failed to fetch');
    const data = await res.json();
    return data;
  } catch (err) {
    console.error('Fetch Error:', err.message);
    throw err;
  }
}
```

- ✓ Always check res.ok
- ✓ Use try...catch
- ✓ Handle loading states
- ✓ Handle empty & error states



7 DEBUGGING LIKE A PRO

Tools & tricks for effective debugging.

- 📄 console.log(), console.table(), console.group()
- 🐞 debugger; statement (pauses execution)
- 🔍 Use breakpoints in DevTools
- 🔗 Check Network, Console & Sources tab
- 🏷️ Use meaningful variable & function names



Good debugging saves hours of guessing!

8 MODERN BEST PRACTICES

- ✓ Use const & let (no var)
- ✓ Prefer arrow functions for simplicity
- ✓ Use === instead of ==
- ✓ Prefer template literals ``
- ✓ Keep functions small & focused
- ✓ Avoid global variables
- ✓ Use early returns to reduce nesting
- ✓ Follow DRY (Don't Repeat Yourself)
- ✓ Write self-explanatory code
- ✓ Comment only when necessary

Clean code is not written by following rules. It's written for humans. 🧑🏻💻

9 COMMON ASYNC PITFALLS

- ✗ Forgetting await (leads to unexpected results)
- ✗ Not handling rejections (UnhandledPromiseRejection)
- ✗ Using async inside Array.forEach()
- ✗ Not returning a Promise in .then()
- ✗ Blocking the main thread (heavy sync tasks)

Fix It:

- ✓ Always await Promises
- ✓ Use for...of instead of forEach with async
- ✓ Keep tasks non-blocking



10 USEFUL CODE SNIPPETS

Delay / Sleep

```
const delay = (ms) =>
  new Promise(res => setTimeout(res, ms));

await delay(1000); // wait 1 sec
```



Retry Function

```
async function retry(fn, retries = 3) {
  try { return await fn(); }
  catch (err) {
    if (retries === 0) throw err;
    await delay(1000);
    return retry(fn, retries - 1);
  }
}
```



Timeout for Promise

```
function timeout(promise, ms) {
  return Promise.race([
    promise,
    new Promise((_, reject) =>
      setTimeout(() => reject(new Error('Timeout')), ms))
  ]);
}
```



11 FINAL TAKEAWAYS

- 🔗 Understand async flow (Call Stack → Event Loop)
- 🔗 Use Promises or Async/Await (not callbacks)
- ✖️ Handle errors everywhere
- 🔗 Write clean, readable & maintainable code
- 🌱 Practice consistently & build real projects
- 🔗 Keep learning & stay updated



INTERVIEW INSIGHT

- Async JS concepts are interview favorites!
- Be ready with callbacks vs promises vs async/await, error handling & event loop.



COMMON MISTAKES

- ✗ Ignoring errors
- ✗ Forgetting await
- ✗ Not validating data
- ✗ Over-engineering simple tasks



BEST PRACTICE

- ✓ Handle errors like a pro
- ✓ Keep code clean & simple
- ✓ Optimize performance
- ✓ Always think about the user



QUICK RECAP

- ✓ Async / Await
- ✓ Promises
- ✓ Error Handling
- ✓ Fetch API
- ✓ Best Practices

REMEMBER

"Simplicity is the soul of efficiency."
- Austin Freeman 😊



SAVE THIS POST
for future revision!



COMMENT "JS"
for full PDF in your DM!



SHARE IT
with your friends!

Follow @abhi_techhub
for more such content!

